



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

13. Hashing



Datenstrukturen – Beispiele

- Grundlegende Datenstrukturen:
 - Kollektion, Stack, Warteschlange, Liste, ...
 - Zahlreiche mögliche Anwendungen
 - Oft Grundlage für speziellere Datenstrukturen
- Indexstrukturen:
 - Bäume und Hash-Tabellen
 - Effizienter Zugriff auf große Datenmengen
- Komplexe Netzwerke:
 - Grundlage: Graphen
 - Verschiedene interne Darstellungsformen
- Anwendungsspezifische Datenstrukturen
 - Elektrische Schaltungen, Genom, Multimedia, ...



Hashverfahren: Prinzip

- Speicherung in Feld 0 bis $N - 1$, einzelne Positionen oft als „buckets“ (Eimer) bezeichnet
- Hashfunktion $h(e)$ bestimmt für Element e die Position im Feld
- h sorgt für „gute“, kollisionsfreie bzw. kollisionsarme Verteilung der Elemente
- letzteres kann nur annäherungsweise durch Gleichverteilung erreicht werden, da Elemente vorher unbekannt!



Beispiel für Hashen

- Beispiel:
 - Array von 0 bis 9, $h(i) = i \bmod 10$
 - Array nach Einfügen von 42 und 119
 - Möglichkeit der Kollision: Versuch des Eintragens von 69

Index	Eintrag
0	
1	
2	42
3	
4	
5	
6	
7	
8	
9	119



Grundlagen: Hashfunktionen

- Hängen vom Datentyp der Elemente und konkreter Anwendung ab
- Für Integer oft $h(i) = i \bmod N$
(funktioniert gut wenn N eine Primzahl ist)
- Für andere Datentypen: Rückführung auf Integer
 - Fließpunkt-Zahlen: Addiere Mantisse und Exponent
 - Strings: Addiere ASCII/Unicode-Werte der/einiger Buchstaben, evtl. jeweils mit Faktor gewichtet
- $h(s) = (a_0 \cdot s[0] + a_1 \cdot s[1] + \dots + a_{l-1} \cdot s[l-1]) \bmod N$



Hashfunktionen: Anforderungen

- Hashfunktionen
 - sollen die Werte gut „streuen“
 - eventuell von Besonderheiten der Eingabewerte abhängig (Buchstaben des Alphabets in Namen tauchen unterschiedlich häufig auf!)
 - sind effizient berechenbar (konstanter Zeitbedarf, auf keinen Fall abhängig von der Anzahl der gespeicherten Werte!)
 - Gleiche Objekte haben gleichen Hash-Wert
 - Hash-Wert ist deterministisch (zum Beispiel nicht zeitabhängig)



Ungünstige Hashfunktionen

- Beispiel #1:
 - $N = 2^i$ und generierte Artikelnummern mit Kontrollziffer 1, 3 oder 7 am Ende
 - $\rightsquigarrow$ Abbildung nur auf ungerade Adressen
- Beispiel #2:
 - Matrikelnummern in Hashtabelle mit 100 Einträgen
 - Variante a: erste beiden Stellen als Hashwert
 - $\rightsquigarrow$ Abbildung auf wenige Buckets
 - Variante b: letzte beiden Stellen
 - $\rightsquigarrow$ gleichmäßige Verteilung



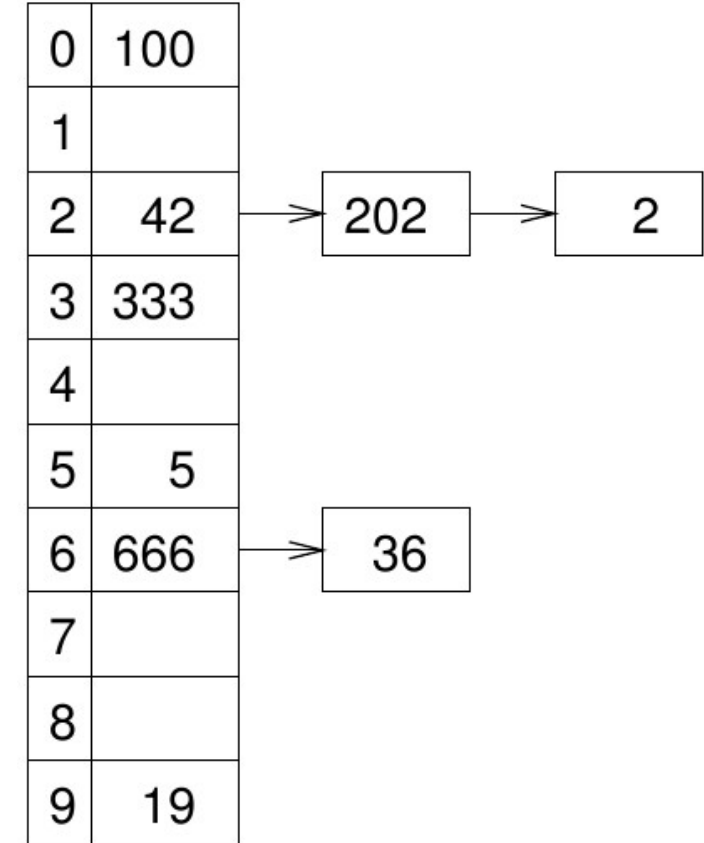
Kollisionsstrategien

- Verkettung der Überläufer bei Kollisionen
 - Verkettete Liste
- Sondieren (Suchen einer alternativen Position)
 - Lineares Sondieren
 - Quadratisches Sondieren
 - doppeltes Hashen, Sondieren mit Zufallszahlen,



Verkettung der Überläufer

- Idee: alle Einträge einer Array-Position werden in einer verketteten Liste verwaltet





Lineares Sondieren

- Idee: falls $T[h(e)]$ besetzt ist, versuche
 - $T[h(e) + 1]$, $T[h(e) + 2]$, $T[h(e) + 3]$, ... $T[h(e) + i]$, ...
 - genauer: $T[(h(e) + 1) \bmod N]$, ...
- Varianten:
 - $T[h(e) + c]$, $T[h(e) + 2c]$, $T[h(e) + 3c]$, ... für Konstante c
 - $T[h(e) + 1]$, $T[h(e) - 1]$, $T[h(e) + 2]$, $T[h(e) - 2]$, ...



Lineares Sondieren

	Leer	Insert 89	Insert 18	Insert 49	Insert 58	Insert 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89



Quadratisches Sondieren

- Idee: falls $T[h(e)]$ besetzt ist, versuche
 - $T[h(e) + 1]$, $T[h(e) + 4]$, $T[h(e) + 9]$, ..., $T[h(e) + i^2]$...
 - jeweils mod N
- Variante:
 - $T[h(e) + 1]$, $T[h(e) - 1]$, $T[h(e) + 4]$, $T[h(e) - 4]$, ...
- wähle Primzahlen für N



Quadratisches Sondieren

	Leer	Insert 89	Insert 18	Insert 49	Insert 58	Insert 69
0				49	49	49
1						
2					58	58
3						69
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89



Aufwand beim Hashen

- bei geringer Kollisionswahrscheinlichkeit:
 - Suchen in $O(1)$
 - Einfügen in $O(1)$
 - Löschen bei Sondiervorgang:
 - nur Markieren der Einträge als gelöscht $O(1)$,
 - oder „Rehashen“ der gesamten Tabelle notwendig $O(n)$
- Füllgrad über 80% : Einfüge- / Suchverhalten wird schnell dramatisch schlechter aufgrund von Kollisionen (bei Sondieren)



Aufwand beim Hashen: Verkettete Überläufer

- Füllgrad (load factor) $\alpha = m/N$
 - m Anzahl gespeicherter Elemente
 - N Anzahl Buckets
- erfolglose Suche (Hashing mit Überlaufliste): $O(1 + \alpha)$
- erfolgreiche Suche ebenfalls in dieser Größenordnung



Aufwand beim Hashen: Sondieren

- Aufwand beim Suchen: $\frac{1}{1-\alpha}$
- typische Werte: $\frac{1}{1-0.5} = 2$, oder $\frac{1}{1-0.9} = 10$
- exakte Analyse basiert auf Aufsummierung der Wahrscheinlichkeit, dass i Elemente den Eimer i belegen (bei Gleichverteilung)
 - Bucket belegt: Wahrscheinlichkeit α
 - Bucket belegt und sondiertes Bucket auch belegt: ungefähr α^2 etc.
 - $1 + \alpha + \alpha^2 + \alpha^3 + \dots = \frac{1}{1-\alpha}$



Klassen `HashMap<K,V>` `Hashtable<K,V>` und in Java

- This implementation provides **constant-time performance for the basic operations** (get and put), assuming the hash function disperses the elements properly among the buckets. Iteration over collection views requires time proportional to the "capacity" of the `HashMap` instance (the number of buckets) plus its size (the number of key-value mappings). Thus, it's very important not to set the initial capacity too high (or the load factor too low) if iteration performance is important.
- An instance of `HashMap` has two parameters that affect its performance: **initial capacity and load factor**. The capacity is the number of buckets in the hash table, and the initial capacity is simply the capacity at the time the hash table is created. The load factor is a measure of how full the hash table is allowed to get before its capacity is automatically increased. When the number of entries in the hash table exceeds the product of the load factor and the current capacity, the hash table is rehashed (that is, internal data structures are rebuilt) so that the hash table has approximately twice the number of buckets.
- As a general rule, the **default load factor (.75)** offers a good tradeoff between time and space costs. Higher values decrease the space overhead but increase the lookup cost (reflected in most of the operations of the `HashMap` class, including get and put). The expected number of entries in the map and its load factor should be taken into account when setting its initial capacity, so as to minimize the number of rehash operations. If the initial capacity is greater than the maximum number of entries divided by the load factor, no rehash operations will ever occur.



Dynamische Hash-Verfahren

- statische Hash-Verfahren
 - fester Bildbereich
 - mehr Einträge erfordern Neuaufbau einer größeren Hash-Tabelle
 - Kopieren aller Elemente
 - Neuberechnung der Hash-Werte aller Elemente!
- dynamische Hash-Verfahren
 - dynamisch wachsende (und schrumpfende) Bildbereiche



Zusammenfassung Hashen

- Berechnung der „Adresse“ eines Eintrags in einer Datenstruktur statt Suchen
- erfordert geeignete Hashfunktion
- mehrfaches Auftreten desselben Hashwertes $\rightsquigarrow$ Kollisionsbehandlung
- Problem: feste Größe des Bildbereichs $\rightsquigarrow$ dynamische Hashverfahren
 - erweiterbares Hashen



Übung Hashen

- Erstellen Sie eine Hash-Tabelle mit $N = 7$ und linearem Sondieren
- Füge ein (in dieser Reihenfolge): 10, 4, 14
- Suche 13, suche 4 (welche Positionen werden betrachtet?)
- Füge weiter ein (in dieser Reihenfolge): 17, 6, 13
- Suche 11, suche 13, suche 15
- Lösche 6
- Suche 13





Übung Hashen

- Erstellen Sie eine Hash-Tabelle mit $N = 7$ und Verkettung der Überläufer
- Füge ein (in dieser Reihenfolge): 10, 4, 14
- Suche 13, suche 4 (welche Positionen werden betrachtet?)
- Füge weiter ein (in dieser Reihenfolge): 17, 6, 13
- Suche 11, suche 13, suche 15
- Lösche 6
- Suche 13



